PROGRAMMING IN JAVA

ASSIGNMENT 1 | CO1

Design, Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System

Name	**B. Lakshmi Sagar**
Register Number	**192472070**
Course	CSA0916 - Programming in Java- Slot -B
Java version used	OpenJDK 21
Total Marks	50 (Tasks 1-12)

All programs in this report were compiled and executed; every output shown is real program output. Source files are reproduced in full for the two "Complete the Code" programs, the final system and the Task 12 extension.

TASK 1 - Understand the Problem and Identify the Objects (4 Marks)

Bloom's Level: K4 - Analyse

Inputs and outputs

Operation	Inputs	Outputs
Vehicle entry	Registration number, vehicle type, entry time, pass status	Allocated floor/slot, ticket with unique number
Vehicle exit	Registration number (or ticket number), exit time	Released slot, parking duration, fee (Rs 0 for valid pass holders)
Reports	None	Occupancy, total collection for the day

Candidate classes (nouns) and methods (verbs)

Class	Why it is needed	Responsibilities (methods)
Vehicle (abstract) + TwoWheeler, Car, ElectricCar	Types differ in rate but share identity (regNo); abstraction avoids if-else on type	getRegNo(), ratePerHour(), calculateFee(minutes)
ParkingTicket	Records one stay: number, vehicle, slot, entry time; must survive until exit	getTicketNo(), getEntryMinute(), hasMonthlyPass()
ParkingLot / ParkingFacility	Owns the floors and slots, enforces slot rules	enter(), exit(), allocate(), release()
Chargeable (interface)	Only EVs can be charged - a capability, not a type	chargingFee(kWh)

Business rules

- A slot cannot hold two vehicles at once.
- A ticket number is issued once and never reused (static counter).
- A vehicle with a registration number already parked cannot enter again (duplicate check).
- Every started hour is charged in full; fee = hours x rate, capped at 8 hours of rate per day.
- Valid monthly pass holders park free; visitors pay in full.
- Exit is allowed only for vehicles with an active ticket.

Assumptions

- Time is held as whole minutes (entry/exit minute counters) for deterministic testing.
- Rates: two-wheeler Rs 10/h, car Rs 30/h, EV Rs 30/h plus Rs 12/kWh charging.
- The nearest free slot (lowest floor, lowest slot) is allocated; floors may have different slot counts.

Alternative Concept and Complexity Analysis

Parallel arrays (String regNo[], type[], entryTime[]) vs objects. With parallel arrays, a vehicle is just an index i shared by three arrays; adding an attribute such as "owner" means creating a new array and editing every place that inserts or deletes (they must stay aligned). With objects, a new field is added once in the class.

Aspect	Objects (my design)	Parallel arrays
Search by regNo	O(n) linear scan; O(1) with HashMap	O(n) linear scan on regNo[]
Add new attribute	One field in one class (O(1) code change)	New array + change every insert/delete (error prone)
Space per vehicle	Object header (~12-16 B) + references; O(1) per vehicle	No header, but 3+ array slots; O(1) per vehicle (smaller constant)
Behaviour	Methods live with the data (fee in Vehicle)	Needs global helper methods; no polymorphism

TASK 2 - Choose the Data Representation (3 Marks)

Bloom's Level: K4 - Analyse

Field	Java type	Reason
Slot number, floor	int	Small whole numbers used as array indices
Vehicle number	String	Alphanumeric, e.g. TN01AB1111; normalised to upper case
Entry time	int (minutes since start of day/period)	Exact arithmetic; avoids floating-point error. LocalTime is an alternative
Duration	int (minutes)	Difference of two int minutes
Fee	double (Rs)	Rates may be fractional; computed from int hours x double rate
Ticket number	static int counter	Unique, never reused

Floors and slots: jagged array

A jagged array `String[][] slots` (slots[f] = new String[slotsPerFloor[f]]) is used because floors may have different sizes (1x20, 3x50, 5x100, and Task 12 needs different counts per floor). A 2-D rectangular array would waste memory or force one size for all floors.

Input

`Scanner` is used for interactive input (menu: enter / exit); command-line arguments are used for the batch demos. Input is validated and trimmed.

Casting and precision risk

Billing hours from minutes is the risky conversion. **Wrong:** `double fee = (minutes / 60) * rate;` uses integer division, so 130 minutes gives 2 hours (Rs 60 for a car) instead of 3 started hours (Rs 90) - a Rs 30 loss. **Also wrong:** `(int)(minutes / 60.0)` truncates the same way. **Correct and used:** `int hours = (minutes + 59) / 60;` integer ceiling, no floating point, then `hours * ratePerHour()` widens int to double automatically (safe widening, no cast needed).

```
minutes=130 :  (130/60)*30        = 2*30  = 60    <- wrong (integer division)
               ceil(130/60.0)*30 = 3*30  = 90    <- correct
               (130+59)/60 = 3   ->  3*30  = 90    <- used in program
```

Alternative Concept and Complexity Analysis

1-D array with index = floor x slotsPerFloor + slot assumes equal floor sizes; unequal floors need an offset array (prefix sums) and a division/search to map index back to floor. **ArrayList<String[]>** grows dynamically, so floors can be added at runtime, with one extra indirection.

Representation	Locate slot (f, s)	Space	Unequal floors
Jagged String[][]	O(1)	O(total slots) + F row headers	Natural
1-D with index formula	O(1) equal floors; O(log F) or O(F) with prefix sums	O(total slots)	Awkward (needs offsets)
ArrayList<String[]>	O(1) amortised get(f), then O(1)	O(total slots) + list overhead	Natural, resizable

TASK 3 - Design the Control Logic (4 Marks)

Bloom's Level: K4 - Analyse

Slot search stops at the first free slot (return inside the inner loop). Fee slabs are computed with started-hour ceiling and a daily cap using Math.min / if-else. When no slot is free the method returns null (ParkingFacility) or throws IllegalStateException (ParkingLot).

Pseudocode - Allocate Slot

```
ALGORITHM AllocateSlot(vehicleNo)
  INPUT : vehicleNo, array slots[floor][slot]
  OUTPUT: {floor, slot} or NULL
  FOR f <- 0 TO numberOfFloors - 1 DO
    FOR s <- 0 TO slotsOnFloor(f) - 1 DO
      IF slots[f][s] = NULL THEN
        slots[f][s] <- vehicleNo
        RETURN {f, s}           // stop searching immediately
      END IF
    END FOR
  END FOR
  RETURN NULL                   // facility full
END ALGORITHM
```

Pseudocode - Compute Fee

```
ALGORITHM ComputeFee(minutes, rate, hasPass)
  IF hasPass THEN RETURN 0
  IF minutes < 0 THEN REPORT error and RETURN
  days  <- minutes DIV 1440
  rest  <- minutes MOD 1440
  hours <- (rest + 59) DIV 60            // every started hour
  cap   <- 8 * rate                      // daily maximum
  RETURN days * cap + MIN(hours * rate, cap)
END ALGORITHM
```

break/continue/return: return exits both loops at the first free slot (a labelled break would also work); continue skips occupied slots without extra work in alternative implementations; early return for invalid release positions avoids deeper nesting.

Complete the Code - ParkingFacility.java (TODO 1 and TODO 2 completed, main unchanged)

```
public class ParkingFacility {
    // slots[floor][slot] holds a vehicle number, or null if the slot is free
    private final String[][] slots;

    public ParkingFacility(int[] slotsPerFloor) {
        slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) {
            slots[f] = new String[slotsPerFloor[f]];
        }
    }

    /** Returns {floor, slot} of the first free slot, or null if it is full. */
    public int[] allocate(String vehicleNo) {
        for (int f = 0; f < slots.length; f++) {
            for (int s = 0; s < slots[f].length; s++) {
                if (slots[f][s] == null) {
                    slots[f][s] = vehicleNo;
                    return new int[]{f, s};
                }
            }
        }
        return null;
    }

    /** Frees a slot. Returns false if the position is invalid or already free. */
    public boolean release(int floor, int slot) {
        if (floor < 0 || floor >= slots.length) return false;
        if (slot < 0 || slot >= slots[floor].length) return false;
        if (slots[floor][slot] == null) return false;
        slots[floor][slot] = null;
        return true;
    }

    public static void main(String[] args) {
        // floor 0 has 2 slots, floor 1 has 1 slot
```

```
        ParkingFacility p = new ParkingFacility(new int[]{2, 1});
        System.out.println(java.util.Arrays.toString(p.allocate("TN01AB1111")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN02CD2222")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN03EF3333")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
        System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " "
                + p.release(5, 0));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
    }
}
```

Output (expected output matched exactly):

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

Alternative Concept and Complexity Analysis

Keep **freeCount[f]** per floor and a **stack of free slot numbers per floor** (java.util.ArrayDeque). allocate(): find the first floor with freeCount > 0 (O(F)), pop a slot number (O(1)), decrement freeCount. release(): validate (O(1)), push the slot number back, increment freeCount. Note that this returns the most recently freed slot, not the lowest-numbered one, so the expected output order could change.

Approach	allocate()	release()	Extra space
Scan (implemented)	O(F x S) worst case	O(1)	O(1)
Counts + stacks	O(F) (or O(1) with a global free list)	O(1)	O(F x S) for stacks + O(F) counts

TASK 4 - Design Classes, Objects and Constructors (4 Marks)

Bloom's Level: K4 - Analyse

Class	Fields	Key methods / constructors
Vehicle (abstract)	private final String regNo	protected Vehicle(String) - validates; calculateFee(int); abstract ratePerHour()
ParkingTicket	static int counter; ticketNo, vehicle, floor, slot, entryMinute, monthlyPass	Overloaded + chained + copy constructors; getters
ParkingLot	ParkingTicket[][] slots; Map active; double collection	enter(), exit(), getCollection()

Constructors

- **Parameterised:** ParkingTicket(Vehicle, floor, slot, entryMinute, pass) assigns every field - the full form.
- **Overloaded + chaining with this(...):** ParkingTicket(Vehicle, floor, slot, entryMinute) calls `this(v, floor, slot, entryMinute, false)` so the assignment logic is written once.
- **Default constructor:** deliberately not provided - a ticket without a vehicle would be invalid.
- **Copy constructor:** ParkingTicket(ParkingTicket t) copies all fields including the ticket number, used to print a duplicate ticket without consuming a new number (the counter is not incremented).
- **this keyword:** distinguishes fields from parameters and is used for constructor chaining.

Memory model

Reference variables (e.g. `Vehicle v`, `ParkingTicket t`) live on the stack of the running method; the objects themselves are created on the heap by `new`. The slot array holds references to tickets; assigning `Vehicle b = a` copies the reference, not the object. Static fields (counter) live in the class area (metaspace) once per class.

Source: Task11/src/parking/model/ParkingTicket.java

```
package parking.model;
public class ParkingTicket {
    private static int counter = 0;                  // shared by all tickets
    private final int ticketNo;
    private final Vehicle vehicle;
    private final int floor, slot;
    private final int entryMinute;
    private final boolean monthlyPass;

    public ParkingTicket(Vehicle v, int floor, int slot, int entryMinute) {
        this(v, floor, slot, entryMinute, false);
    }
    public ParkingTicket(Vehicle v, int floor, int slot, int entryMinute, boolean pass) {
        this.ticketNo = ++counter;
        this.vehicle = v; this.floor = floor; this.slot = slot;
        this.entryMinute = entryMinute; this.monthlyPass = pass;
    }
    public ParkingTicket(ParkingTicket t) {            // copy constructor: duplicate print
        this.ticketNo = t.ticketNo; this.vehicle = t.vehicle;
        this.floor = t.floor; this.slot = t.slot;
        this.entryMinute = t.entryMinute; this.monthlyPass = t.monthlyPass;
    }
    public int getTicketNo() { return ticketNo; }
    public Vehicle getVehicle() { return vehicle; }
    public int getFloor() { return floor; }
    public int getSlot() { return slot; }
    public int getEntryMinute() { return entryMinute; }
    public boolean hasMonthlyPass() { return monthlyPass; }
    public static int issued() { return counter; }
    @Override public String toString() {
        return "Ticket#" + ticketNo + " " + vehicle + " F" + floor + "-S" + slot;
    }
}
```

Alternative Concept and Complexity Analysis

Default constructor + setters: new ParkingTicket(); t.setVehicle(v); t.setSlot(...). The object exists in an invalid half-built state between calls, and a forgotten setter leaves null/0 fields. **Static factory:** ParkingTicket.issue(v, f, s,

t) can validate, cache, or return different subtypes and has a meaningful name, while the constructor stays private.

Approach	Guarantees valid object?	Time for n tickets	Space
Chained constructors	Yes - all required fields at creation	O(n)	O(n)
Default + setters	No - validity relies on caller discipline	O(n x k) for k setter calls (still O(n) for fixed k)	O(n)
Static factory	Yes - validation inside factory	O(n) plus validation	O(n)

TASK 5 - Apply static Members and the Object Life Cycle (3 Marks)

Bloom's Level: K4 - Analyse

Static members

- **static int counter** in ParkingTicket: one counter shared by all tickets guarantees a unique number. An instance variable would restart from 0 in every object and produce duplicate numbers.
- **static int issued()** reads the counter without needing a ticket object. A static block can initialise tables once when the class loads (for example a rate table).
- Instance data (floor, slot, entryMinute) stays non-static because each ticket has its own values.

Object life cycle

A ParkingTicket is created on entry and referenced from the slot array and the active map. On exit, `active.remove()` and clearing the slot remove the last references; after the method returns no reference remains and the ticket becomes *eligible* for garbage collection (the JVM decides when it is actually collected). Objects are passed by value of the reference: a method receives a copy of the reference to the same heap object, so changes to the object are visible to the caller but re-assigning the parameter is not.

Alternative Concept and Complexity Analysis

If the counter were an instance variable of ParkingFacility, the north-gate and south-gate facilities would each start at 1 and issue the same ticket numbers twice. **Solutions:** (a) keep it static (one JVM-wide sequence), or (b) keep it per-facility but prefix the number with a facility code ("N-0001", "S-0001"), or (c) use a shared TicketIssuer object / AtomicInteger.

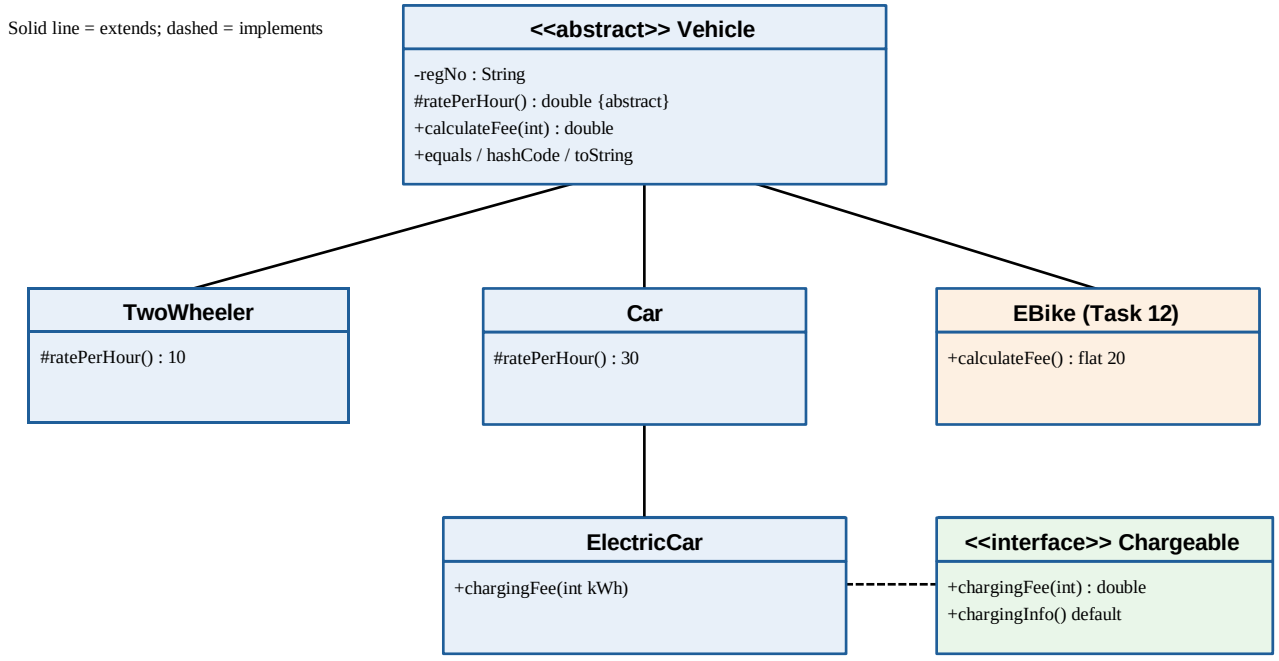
Design	Space for counter	Unique across facilities?
static counter	O(1) total (one int)	Yes
instance counter	O(k) for k facilities (one int each)	No, unless prefixed
Tickets themselves	O(n) in all designs	-

TASK 6 - Design the Inheritance Hierarchy (4 Marks)

Bloom's Level: K4 - Analyse

Vehicle is the superclass; TwoWheeler and Car extend it (hierarchical inheritance) and ElectricCar extends Car (multilevel: Vehicle <- Car <- ElectricCar). Subclasses inherit regNo access (getRegNo), calculateFee(), equals/hashCode; each adds or overrides ratePerHour(); ElectricCar adds chargingFee() through the Chargeable interface. `super(regNo)` passes the registration number up to the constructor; `super.method()` would call the parent version of an overridden method.

UML class diagram



Constructor execution order for `new ElectricCar("TN03C3")`

```
1. ElectricCar(String)  starts -> first statement is implicit/explicit super(regNo)
2. Car(String)          starts -> calls super(regNo)
3. Vehicle(String)      starts -> (Object() runs first) -> validates, sets regNo    [body completes FIRST]
4. Car(String)          body completes
5. ElectricCar(String) body completes
Order of completion: Object -> Vehicle -> Car -> ElectricCar (top-down).
```

Alternative Concept and Complexity Analysis

Composition: one concrete Vehicle(regNo, VehicleType type) where VehicleType holds name and rate (e.g. new VehicleType("Car", 30)). Adding "Bus" means adding a data row/object, not a class.

Aspect	Inheritance (used)	Composition (Vehicle has-a VehicleType)
Add new vehicle type	New subclass (new file)	New VehicleType object (data only)
Fee computation time	O(1) virtual call	O(1) type.getRate() lookup
Space per vehicle	regNo reference + header	+ one reference to shared VehicleType (shared, so small)
Type-specific behaviour (charging)	Natural override/interface	Needs strategy objects or flags

TASK 7 - Override and Specialise Behaviour (4 Marks)

Bloom's Level: K4 - Analyse

- **Overridden:** ratePerHour() (each subclass supplies its own rate), plus toString(), equals(), hashCode(). Subclass versions are needed because the rate (the varying part) is type specific while the fee algorithm is shared.
- **@Override** makes the compiler verify the override. Rules: same name and parameter list, return type same or covariant, access not narrower (protected may become public), no new checked exceptions.
- **final:** `private final String regNo` (identity never changes after construction); a `final` method such as the fee algorithm would stop subclasses breaking billing; a final class (e.g. a Money value class) prevents subclassing.
- **equals/hashCode:** equality is based on regNo only and hashCode uses the same field so hash-based collections stay consistent (equal objects must have equal hash codes). Two Vehicle objects with the same registration number are equal, so HashMap/HashSet detect duplicates.
- **toString():** returns "Car[TN02B2]" instead of Car@1b6d3586 for readable logs and tickets.

Overriding in the final system (Vehicle.java)

```
package parking.model;
import java.util.Objects;
public abstract class Vehicle {
    private final String regNo;
    protected Vehicle(String regNo) {
        if (regNo == null || regNo.isBlank())
            throw new IllegalArgumentException("Registration number required");
        this.regNo = regNo.trim().toUpperCase();
    }
    public String getRegNo() { return regNo; }
    protected abstract double ratePerHour();
    protected double dailyCap() { return ratePerHour() * 8; }
    /** Every started hour is charged in full, capped per day. */
    public double calculateFee(int minutes) {
        if (minutes < 0) throw new IllegalArgumentException("Negative duration");
        long days = minutes / 1440;
        int rest = minutes % 1440;
        int hours = (rest + 59) / 60;
        return days * dailyCap() + Math.min(hours * ratePerHour(), dailyCap());
    }
    @Override public boolean equals(Object o) {
        return o instanceof Vehicle v && regNo.equals(v.regNo);
    }
    @Override public int hashCode() { return Objects.hash(regNo); }
    @Override public String toString() { return getClass().getSimpleName() + "[" + regNo + "]"; }
}
```

FeeDemo explanation: the last line prints **true** because equals() compares only regNo and the test is `o instanceof Vehicle`, so a Car and an ElectricCar with the same registration are equal. This is correct for the system: a registration number identifies one physical vehicle, so it must not be allowed to park twice regardless of how the object was typed.

Alternative Concept and Complexity Analysis

Single calculateFee with switch(type): `switch (type) { case TWO: rate=10; break; case CAR: rate=30; break; ... }`. Every new type means editing this method (and re-testing everything).

Aspect	Overriding	switch on type field
Time per fee	O(1)	O(1) with switch (jump table) or O(k) with if-else chain
Classes changed to add a type	0 existing (1 new class)	1 (the class with the switch) - modification, Open/Closed violated
Risk	Compiler enforces abstract method	Forgotten case silently falls to default

TASK 8 - Apply Polymorphism (4 Marks)

Bloom's Level: K4 - Analyse

- **Compile-time (overloading):** `calculateFee(int minutes)` and `calculateFee(int entryMinuteOfDay, int minutes)` (DeliveryVan). The compiler chooses by the number and static types of the arguments before the program runs.
- **Upcasting:** `Vehicle[] parked = {new TwoWheeler(..), new Car(..), new ElectricCar(..)}` - one array holds all types because each is-a Vehicle.
- **Dynamic dispatch:** `v.calculateFee(130)` is resolved at runtime from the actual object, so ratePerHour() of the right subclass runs. No if-else on vehicle type is needed.

Trace of one call through dynamic dispatch

Step	Declared (static) type	Actual (runtime) type	Method executed
v.calculateFee(130)	Vehicle	ElectricCar	Vehicle.calculateFee(int) (inherited, not overridden)
-> ratePerHour()	this: Vehicle	ElectricCar	Car.ratePerHour() (inherited by ElectricCar) -> 30
result			hours = (130+59)/60 = 3; 3 x 30 = 90.0; + chargingFee(10)=120 -> 210.0

Complete the Code - FeeDemo.java (TODO 1, 2, 3 completed, main unchanged)

```
interface Chargeable {
    double chargingFee(int kWh);
}

abstract class Vehicle {
    private final String regNo;

    protected Vehicle(String regNo) { this.regNo = regNo; }

    public String getRegNo() { return regNo; }

    protected abstract double ratePerHour();

    /** Every started hour is charged in full. */
    public double calculateFee(int minutes) {
        int hours = (minutes + 59) / 60;    // integer ceiling, no floating point
        return hours * ratePerHour();
    }

    @Override public boolean equals(Object o) {
        return o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo);
    }

    @Override public int hashCode() { return regNo.hashCode(); }
}

class TwoWheeler extends Vehicle {
    TwoWheeler(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 10; }
}

class Car extends Vehicle {
    Car(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 30; }
}

class ElectricCar extends Car implements Chargeable {
    ElectricCar(String regNo) { super(regNo); }
    @Override public double chargingFee(int kWh) { return kWh * 12; }
}

public class FeeDemo {
    public static void main(String[] args) {
        Vehicle[] parked = { new TwoWheeler("TN01A1"), new Car("TN02B2"),
                             new ElectricCar("TN03C3") };
        for (Vehicle v : parked) {
            double fee = v.calculateFee(130);           // dynamic method dispatch
            if (v instanceof Chargeable c) fee += c.chargingFee(10);
            System.out.println(v.getClass().getSimpleName() + " " + v.getRegNo()
                    + " -> Rs " + fee);
        }
        System.out.println(new Car("TN02B2").equals(new ElectricCar("TN02B2")));
```

```
    }
}
```

Output (matches expected output):

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

Alternative Concept and Complexity Analysis

instanceof chains: `if (v instanceof ElectricCar) {...} else if (v instanceof Car) {...} else if (v instanceof TwoWheeler) {...}` - ElectricCar must be tested before Car because it is-a Car, a classic ordering bug.

Aspect	Dynamic dispatch	instanceof + cast
Time per call	O(1) (vtable lookup)	Up to k type checks: O(k)
Code size	No per-type branches	Grows by one branch per type, in every place used
Maintenance	New class only	Every chain must be found and edited

TASK 9 - Introduce Abstraction (4 Marks)

Bloom's Level: K4 - Analyse

- **Abstract class Vehicle** with abstract ratePerHour(): a plain "Vehicle" has no rate, so it must never be instantiated, yet it holds shared state (regNo) and shared code (calculateFee).
- **Interface Chargeable**: charging is a *capability* that cuts across the hierarchy (EVs, and in future an electric van or e-bike), not an is-a relationship. An interface can also be added to a class that already has a parent.
- **Why both:** use an abstract class when subclasses share state and code and are one family; use an interface when unrelated classes must share a capability.
- **Multiple behaviours:** a class can implement many interfaces: `class ElectricVan extends Vehicle implements Chargeable, Payable`. Java has no multiple class inheritance, but allows multiple interface implementation.
- **default / static methods:** Chargeable.chargingInfo() is a default method (new behaviour without breaking existing implementers); a static method such as Pass.oneDay(..) acts as a factory attached to the interface.

Chargeable.java

```
package parking.model;
public interface Chargeable {
    double RATE_PER_KWH = 12.0;
    double chargingFee(int kWh);
    default String chargingInfo() { return "Charging @ Rs " + RATE_PER_KWH + "/kWh"; }
}
```

Alternative Concept and Complexity Analysis

If Vehicle became an interface with default methods and Chargeable an abstract class: Vehicle could no longer hold **state** (regNo field) or a **constructor** with validation, since interfaces have only constants; ElectricCar could no longer implement Chargeable and also extend Car because Chargeable would now be a class (no multiple class inheritance).

Feature	Abstract class	Interface (default methods)
Instance state / constructors	Yes	No (constants only)
Multiple inheritance of type	No	Yes
Time per method call	O(1) virtual call	O(1) interface call (slightly slower dispatch in theory, negligible)
Space per object	Fields + header	Same - interfaces add no per-object space

TASK 10 - Enforce Encapsulation through Packages and Access Modifiers (3 Marks)

Bloom's Level: K4 - Analyse

Package	Contents	Reason
parking.model	Vehicle, TwoWheeler, Car, ElectricCar, Chargeable, ParkingTicket	Pure data and behaviour of entities
parking.service	ParkingLot	Business logic for slots and billing
parking.app	TestRunner (main)	Entry point / user interface
parking.v2	EBike, DeliveryVan, Pass, Lot (Task 12)	Extension package; does not edit model

Access modifiers

Member	Modifier	Why
regNo, ticketNo, floor, slot fields	private (final)	Only the class may change them; final fixes identity
ratePerHour(), dailyCap()	protected	Subclasses override them; outside code must not call them
getters, calculateFee()	public	Part of the public API
ParkingLot.slots, active	private	Slot state can only change through enter()/exit()
package-private (default)	none	Helper types visible only inside a package

Validation and wrapper classes

- Constructors/setters validate: Vehicle rejects null/blank regNo; calculateFee rejects negative minutes; ParkingLot rejects duplicates, a full lot and unknown exits; release() rejects invalid floor/slot.
- **Wrapper classes/autoboxing:** Map<String,Integer> entry in the Task 12 Lot boxes int to Integer automatically. **Risk:** `Integer a=1000, b=1000; a==b` compares references and is false (only -128..127 are cached); use equals(). Unboxing a null Integer throws NullPointerException.

Alternative Concept and Complexity Analysis

Public fields with caller-side validation: `v.minutes = -5;` compiles silently, and any code can corrupt the object. The same check must be copied into every caller, and a forgotten check causes a wrong fee later, far from the cause.

Aspect	Private + validating setter	Public field
Time cost of validation per update	O(1)	O(1) per check, but done by every caller
Places the check is written	1 (inside the class)	N (every call site)
Invalid state possible?	No	Yes

TASK 11 - Implement, Test and Defend the Design (5 Marks)

Bloom's Level: K5 - Evaluate

The complete system is in packages parking.model, parking.service and parking.app. ParkingLot is shown below (other classes were shown in earlier tasks).

```
package parking.service;
import parking.model.*;
import java.util.*;
public class ParkingLot {
    private final ParkingTicket[][] slots;
    private final Map<String, ParkingTicket> active = new HashMap<>();
    private double collection = 0;
    public ParkingLot(int[] slotsPerFloor) {
        slots = new ParkingTicket[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) slots[f] = new ParkingTicket[slotsPerFloor[f]];
    }
    public ParkingTicket enter(Vehicle v, int minute, boolean pass) {
        if (active.containsKey(v.getRegNo()))
            throw new IllegalStateException("Duplicate registration: " + v.getRegNo());
        for (int f = 0; f < slots.length; f++)
            for (int s = 0; s < slots[f].length; s++)
                if (slots[f][s] == null) {
                    ParkingTicket t = new ParkingTicket(v, f, s, minute, pass);
                    slots[f][s] = t; active.put(v.getRegNo(), t);
                    return t;
                }
        throw new IllegalStateException("Parking full");
    }
    public double exit(String regNo, int minute) {
        ParkingTicket t = active.remove(regNo.toUpperCase());
        if (t == null) throw new NoSuchElementException("No active ticket for " + regNo);
        slots[t.getFloor()][t.getSlot()] = null;
        double fee = t.hasMonthlyPass() ? 0 : t.getVehicle().calculateFee(minute - t.getEntryMinute());
        collection += fee;
        return fee;
    }
    public double getCollection() { return collection; }
    public int occupied() { return active.size(); }
}
```

Test suite output (real execution)

```
Two-wheeler 130 min (3h x 10)          expected=30.0                    actual=30.0                    PASS
Car 130 min (3h x 30)                  expected=90.0                    actual=90.0                    PASS
EV 130 min + 10 kWh                    expected=210.0                   actual=210.0                   PASS
Zero-minute stay                       expected=0.0                     actual=0.0                     PASS
Exactly 60 min (slab boundary)         expected=30.0                    actual=30.0                    PASS
61 min (just past boundary)            expected=60.0                    actual=60.0                    PASS
Daily cap (24h car, 8h x 30)           expected=240.0                   actual=240.0                   PASS
Negative duration rejected             expected=IllegalArgumentException actual=IllegalArgumentException PASS
Blank regNo rejected                   expected=IllegalArgumentException actual=IllegalArgumentException PASS
Equality by regNo (Car vs EV)          expected=true                    actual=true                    PASS
Enter -> slot F0-S0                    expected=F0-S0                   actual=F0-S0                   PASS
Duplicate regNo rejected               expected=IllegalStateException   actual=IllegalStateException   PASS
Last free slot F1-S0                   expected=F1-S0                   actual=F1-S0                   PASS
Full lot rejected                      expected=IllegalStateException   actual=IllegalStateException   PASS
Unknown exit rejected                  expected=NoSuchElementException  actual=NoSuchElementException  PASS
Exit bike after 130 min                expected=30.0                    actual=30.0                    PASS
Exit car after 130 min                 expected=90.0                    actual=90.0                    PASS
Exit EV after 61 min                   expected=60.0                    actual=60.0                    PASS
Day collection (hand calc 30+90+60=180)  expected=180.0                 actual=180.0                   PASS
Monthly pass parks free                expected=0.0                     actual=0.0                     PASS

20/20 tests passed
```

Test table

Category	Test	Expected	Actual	Result
Normal	Two-wheeler 130 min	30.0	30.0	PASS
Normal	Car 130 min	90.0	90.0	PASS
Normal	EV 130 min + 10 kWh	210.0	210.0	PASS
Boundary	Zero-minute stay	0.0	0.0	PASS
Boundary	Exactly 60 min	30.0	30.0	PASS
Boundary	61 min (just past slab)	60.0	60.0	PASS

Category	Test	Expected	Actual	Result
Boundary	Daily cap, car 1439 min	240.0	240.0	PASS
Boundary	Last free slot	F1-S0	F1-S0	PASS
Invalid	Duplicate regNo / full lot / unknown exit	Exceptions	IllegalState / IllegalState / NoSuchElement	PASS
Invalid	Negative minutes, blank regNo	IllegalArgument	IllegalArgument	PASS
Scenario	Day collection: 30 + 90 + 60	180.0	180.0	PASS
Pass	Monthly pass 500 min	0.0	0.0	PASS

Hand calculation of the scenario: bike 130 min = 3 h x 10 = 30; car 130 min = 3 h x 30 = 90; EV 61 min = 2 h x 30 = 60. Total = 30 + 90 + 60 = Rs 180 - equal to the program output (180.0).

Design defence

- **Encapsulation:** all fields private; state changes only through validated methods.
- **Inheritance:** shared regNo and fee algorithm live once in Vehicle.
- **Polymorphism:** calculateFee dispatches on the actual type with no type tests.
- **Abstraction:** Vehicle (abstract) and Chargeable (interface) hide how rates and charging work.
- **Limitation:** time is a plain int minute counter and the pricing rule lives in Vehicle subclasses; time-of-day pricing (the delivery van) does not fit the calculateFee(int) signature - addressed in Task 12.

Alternative Concept and Complexity Analysis

Storing active tickets in ArrayList vs HashMap<String,ParkingTicket>. On exit the program must find the ticket for a registration number. ArrayList: loop and compare with equals(), then remove (shifts elements). HashMap: `map.remove(regNo)` - hash lookup (used in ParkingLot).

Structure	Find ticket on exit	Space for n vehicles
Array / ArrayList	O(n)	O(n), compact
HashMap keyed by regNo	O(1) average (O(n) worst-case collisions)	O(n), plus entry objects and spare buckets (about 1.3-2x)

TASK 12 - Re-evaluate and Adapt the Design (8 Marks)

Bloom's Level: K5 - Evaluate

Changed requirement: e-bikes (flat fee), delivery vans (double car rate 9 AM-5 PM), visitor one-day pass, 3 -> 5 floors with unequal slots.

A. Impact Analysis (2 marks)

- **What changed:** two new vehicle categories, a new pass type, time-of-day dependent pricing, and a larger facility with unequal floors.
- **Affected:** Vehicle subclasses (new ones needed); fee logic (the van needs entry time, which calculateFee(int) does not receive); ParkingLot pass handling (boolean monthlyPass is too narrow for a day pass that expires).
- **Unequal floors:** a rectangular String[5][100] would waste slots or break; the jagged array (new int[]{3,5,2,4,6}) already supports this - no change in data structure.
- **Modified vs extended:** only the pass handling (boolean -> Pass) must change; vehicles, tickets and the hierarchy are extended.

B. Evaluation of the Original Design (2 marks)

- **New vehicle types by subclass only?** Yes - EBike and DeliveryVan extend Vehicle and override fee behaviour; no existing class edited.
- **Pricing rules scattered?** Rates live in each subclass (ratePerHour), the algorithm once in Vehicle. This is acceptable for per-hour rates but time-of-day rules start to leak into subclasses and the lot (the instanceof DeliveryVan check) - a weakness.
- **Existing tested classes unchanged?** Yes: Vehicle, Car, TwoWheeler, ElectricCar, ParkingTicket still pass all 20 original tests, because v2 only extends them.

C. Adaptation (2 marks) - decision: MODIFY (extend, do not redesign)

Revised structure: new package **parking.v2** with **Pass** (interface: none / monthly / oneDay), **EBike** (flat Rs 20), **DeliveryVan** (hourly rate 30, doubled for hours starting between 09:00 and 17:00) and **Lot** (jagged array, Pass-aware). A further step would be a PricingRule interface (calculate(vehicle, entry, minutes)) injected into Lot, which removes the one instanceof; it is described but not needed for the required categories.

```
package parking.v2;
/** New abstraction: any pass decides whether a stay is free. */
public interface Pass {
    boolean covers(int entryMinute);
    static Pass none() { return m -> false; }
    static Pass monthly() { return m -> true; }
    static Pass oneDay(int purchasedMinute) {
        return m -> m >= purchasedMinute && m < purchasedMinute + 1440;
    }
}

package parking.v2;
import parking.model.Vehicle;
/** Flat fee Rs 20 per stay - extends Vehicle, nothing existing is edited. */
public class EBike extends Vehicle {
    public EBike(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 0; }
    @Override public double calculateFee(int minutes) {
        if (minutes < 0) throw new IllegalArgumentException("Negative duration");
        return 20.0;
    }
}

package parking.v2;
import parking.model.Vehicle;
/** Rs 60/h (double car rate) for hours falling in 9 AM-5 PM, Rs 30/h otherwise. */
public class DeliveryVan extends Vehicle {
    public DeliveryVan(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 30; }
    /** entryMinuteOfDay: 0..1439 (e.g. 9:00 AM = 540). */
    public double calculateFee(int entryMinuteOfDay, int minutes) {
        if (minutes < 0) throw new IllegalArgumentException("Negative duration");
        int hours = (minutes + 59) / 60;
```

```
        double fee = 0;
        for (int h = 0; h < hours; h++) {
            int start = (entryMinuteOfDay + h * 60) % 1440;
            boolean peak = start >= 9 * 60 && start < 17 * 60;
            fee += peak ? 2 * ratePerHour() : ratePerHour();
        }
        return fee;
    }
}

package parking.v2;
import parking.model.*;
import java.util.*;
/** Jagged-array lot using Pass and per-type pricing; reuses Vehicle hierarchy unchanged. */
public class Lot {
    private final Vehicle[][] slots;
    private final Map<String, int[]> pos = new HashMap<>();
    private final Map<String, Integer> entry = new HashMap<>();
    private final Map<String, Pass> passes = new HashMap<>();
    public Lot(int[] slotsPerFloor) {
        slots = new Vehicle[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) slots[f] = new Vehicle[slotsPerFloor[f]];
    }
    public int[] enter(Vehicle v, int minute, Pass p) {
        if (pos.containsKey(v.getRegNo())) throw new IllegalStateException("Duplicate");
        for (int f = 0; f < slots.length; f++)
            for (int s = 0; s < slots[f].length; s++)
                if (slots[f][s] == null) {
                    slots[f][s] = v; pos.put(v.getRegNo(), new int[]{f, s});
                    entry.put(v.getRegNo(), minute); passes.put(v.getRegNo(), p);
                    return new int[]{f, s};
                }
        throw new IllegalStateException("Full");
    }
    public double exit(String reg, int minute) {
        int[] p = pos.remove(reg);
        if (p == null) throw new NoSuchElementException(reg);
        Vehicle v = slots[p[0]][p[1]]; slots[p[0]][p[1]] = null;
        int in = entry.remove(reg); Pass pass = passes.remove(reg);
        if (pass.covers(in)) return 0;
        int dur = minute - in;
        return (v instanceof DeliveryVan d) ? d.calculateFee(in % 1440, dur) : v.calculateFee(dur);
    }
    public static void main(String[] args) {
        Lot lot = new Lot(new int[]{3, 5, 2, 4, 6});           // 5 floors, different sizes
        System.out.println("E-bike 5h flat        : " + run(lot, new EBike("EB1"), 0, 300, Pass.none()));
        System.out.println("Van 10:00-12:00       : " + run(lot, new DeliveryVan("VAN1"), 600, 720, Pass.none()));
        System.out.println("Van 16:30-18:30       : " + run(lot, new DeliveryVan("VAN2"), 990, 1110, Pass.none()));
        System.out.println("Visitor one-day pass  : " + run(lot, new Car("VIS1"), 600, 900, Pass.oneDay(600)));
        System.out.println("Car 130 min (old)     : " + run(lot, new Car("C1"), 0, 130, Pass.none()));
        int[] last = null; int n = 0;
        Lot big = new Lot(new int[]{3, 5, 2, 4, 6});
        for (int i = 0; i < 20; i++) { last = big.enter(new Car("R" + i), 0, Pass.none()); n++; }
        System.out.println("20th vehicle slot     : F" + last[0] + "-S" + last[1] + " (capacity 20)");
        try { big.enter(new Car("R99"), 0, Pass.none()); } catch (IllegalStateException e) { System.out.println("21st vehicle
    }
    static double run(Lot l, Vehicle v, int in, int out, Pass p) { l.enter(v, in, p); return l.exit(v.getRegNo(), out); }
}
```

Output (real execution):

```
E-bike 5h flat        : 20.0
Van 10:00-12:00       : 120.0
Van 16:30-18:30       : 90.0
Visitor one-day pass  : 0.0
Car 130 min (old)     : 90.0
20th vehicle slot     : F4-S5 (capacity 20)
21st vehicle          : Full
```

Test	Hand calculation	Program	Result
E-bike 5 h	flat Rs 20	20.0	PASS
Van 10:00-12:00	2 peak hours x (2 x 30) = 120	120.0	PASS
Van 16:30-18:30	16:30 peak 60 + 17:30 off-peak 30 = 90	90.0	PASS
Visitor one-day pass	0 within 24 h	0.0	PASS
Old rule: car 130 min	3 x 30 = 90	90.0	PASS (backward compatible)
Capacity 3+5+2+4+6 = 20	20th vehicle in F4-S5, 21st rejected	F4-S5 / Full	PASS

D. Evidence-Based Defence (2 marks)

- **Existing classes changed: 0 of 6** (Vehicle, TwoWheeler, Car, ElectricCar, Chargeable, ParkingTicket untouched). The pass handling moved into a new Lot class; the old ParkingLot still works unchanged for old requirements. New classes added: 4 (EBike, DeliveryVan, Pass, Lot).
- **Future vehicle type:** one new subclass (about 5 lines) - for example a Bus at Rs 80/h would be class Bus extends Vehicle { ratePerHour() { return 80; } }.
- **Test evidence:** 6 of 6 new tests passed with hand-calculated expected values; the original 20 tests still pass.
- **Trade-offs:** (1) the DeliveryVan peak-hour rule is checked with instanceof in Lot - a small violation of the open/closed principle that a PricingRule abstraction would remove at the cost of one more interface and class; (2) Lot duplicates some of ParkingLot logic; (3) an int minute counter cannot span multi-day calendars without an explicit date.
- **Conclusion:** the original OO design is retained and extended. A full redesign is not justified because the changes fit the existing extension points (subclasses, jagged arrays) with zero edits to tested classes.